



LeetCode 1920: Build Array from Permutation

1. Problem Title & Link

Title: LeetCode 1920: Build Array from Permutation

Link: <https://leetcode.com/problems/build-array-from-permutation/>

2. Problem Statement (Short Summary)

You are given a zero-based permutation array `nums`.

Create a new array `ans` such that:

`ans[i] = nums[nums[i]]`

Return the new array.

A permutation means every number from 0 to `n-1` appears exactly once.

3. Examples (Input → Output)

Example 1

Input:

`nums = [0,2,1,5,3,4]`

Output:

`[0,1,2,4,5,3]`

Explanation:

`ans[0] = nums[nums[0]] = nums[0] = 0`

`ans[1] = nums[nums[1]] = nums[2] = 1`

`ans[2] = nums[nums[2]] = nums[1] = 2`

...

Example 2

Input:

`nums = [5,0,1,2,3,4]`

Output:

`[4,5,0,1,2,3]`

Example 3

Input:

`nums = [0]`

Output:

`[0]`

4. Constraints



1 <= nums.length <= 1000
0 <= nums[i] < nums.length
nums is a permutation
Extra Space Allowed: Yes

5. Core Concept (Pattern / Topic)

Array Manipulation

6. Thought Process (Step-by-Step Explanation)

Brute Force

For every index:

1. Find nums[i]
2. Use it as another index
3. Store nums[nums[i]]

Since direct indexing is $O(1)$, this solution is already optimal.

Optimized Thinking

Because permutation guarantees valid indices:

nums[i] → another index

Simply create answer array and fill values.

7. Visual / Intuition Diagram (ASCII Diagram)

```
nums

Index : 0 1 2 3 4 5
Value : 0 2 1 5 3 4

i = 3

nums[3] = 5

ans[3] = nums[5]

      ↓
Value = 4
```

8. Pseudocode (Language Independent)

```
create ans

for i from 0 to n-1
```



```
ans[i] = nums[nums[i]]

return ans
```

9. Code Implementation

Python

```
class Solution:
    def buildArray(self, nums):
        ans = []

        for i in range(len(nums)):
            ans.append(nums[nums[i]])

        return ans
```

Java

```
class Solution {
    public int[] buildArray(int[] nums) {

        int[] ans = new int[nums.length];

        for(int i = 0; i < nums.length; i++) {
            ans[i] = nums[nums[i]];
        }

        return ans;
    }
}
```

10. Time & Space Complexity

Time Complexity : $O(n)$

Space Complexity : $O(n)$

11. Common Mistakes / Edge Cases

Mistakes

Using `nums[i]` instead of `nums[nums[i]]`

Modifying `nums` before accessing required values

Edge Cases

Single element array

`nums = [0]`



12. Detailed Dry Run

Input:

nums = [0,2,1,5,3,4]

i	nums[i]	nums[nums[i]]	ans
0	0	0	[0]
1	2	1	[0,1]
2	1	2	[0,1,2]
3	5	4	[0,1,2,4]
4	3	5	[0,1,2,4,5]
5	4	3	[0,1,2,4,5,3]

Output:

[0,1,2,4,5,3]

13. Common Use Cases (Real-Life / Interview)

Index remapping

Graph node reordering

Data transformation

Lookup table generation

14. Common Traps (Important!)

Confusing value with index

Using nested loops unnecessarily

15. Builds To (Related LeetCode Problems)

LC 1920 - Build Array from Permutation

LC 1929 - Concatenation of Array

LC 1480 - Running Sum of 1D Array

LC 867 - Transpose Matrix

16. Alternate Approaches + Comparison

Approach	Time	Space
Extra Array	O(n)	O(n)
In-place Encoding	O(n)	O(1)



17. Why This Solution Works (Short Intuition)

Each element already represents a valid index. Directly accessing `nums[nums[i]]` gives the required mapped value.

18. Variations / Follow-Up Questions

1. Can you solve using $O(1)$ extra space?
2. What if `nums` is not a permutation?
3. Can this be done in-place?
4. What if values repeat?